

have a terminal open, the `locate` command can help find it quickly, and a copy/paste will get it going in no time:

```
$ locate run-emulator
/usr/google/appinventor-extras/commands-for-Appinventor/run-emulator
$ /usr/google/appinventor-extras/commands-for-Appinventor/run-emulator
$ emulator: WARNING: system partition size adjusted to match image file (74 MB > 66 MB)
emulator: warning: opening audio input failed
```

The emulator, which is based on QEMU, appears in a new window. It takes some time to start—perhaps even a minute or more—because it’s booting up the virtual phone ‘from scratch’. While booting, it displays first the plain text ‘Android’, then a stylised and animated Android logo, finally bringing up the Android desktop. In Figure 8, the Blocks Editor is connected to the emulator, and the MoleMash game tutorial is running. You can see the simulated buttons and keypad on the right of the window.

As you work, you’ll need to switch between the emulator window, the Blocks Editor, and the Designer. The App Inventor team warns: “Don’t connect a physical phone to the computer while you’re using the emulator, or App Inventor will become confused. If you want to switch to testing on a real phone, it would be best to close the Blocks Editor, shut down the emulator, plug in the phone, check that you can find it using `adb` (see the boxes on the ‘set-up’ and on ‘re-connecting’) and then re-open the Blocks Editor.”

Observations

Here are some of my observations while using this experimental service. They are not complaints; I expect that as the App Inventor team continues working, these and other minor glitches will be fixed, and the service will turn out to be a real asset to Google, as well as to people looking to quickly develop applications for Android.

Typelocking is great!

At some point during the tutorials, you’re introduced to the typelocking feature of the Blocks Editor, where you start typing the name of a component or variable, and get a pop-up drop-down list containing possible completions. While using this feature, the pop-up frame is way too narrow for even the component names you are advised to use in the tutorials, i.e., it can sometimes be plain guess-work, leaving you unsure whether you’re choosing the right one or not. Still, once you select a component and hit *Enter*, the block is placed on the canvas for you—and if you had recently worked on a block whose socket accepts the type of this new block, the Blocks Editor obligingly snaps it home in the socket for you! Neat!

Another feature that saves visiting the blocks drawers for built-in blocks is to left-click on the canvas. This presents you with a

menu, from which you can quickly select built-in blocks, as shown in Figure 9. In this case, I needed a colour block, and picked *Red* from the *Colors* menu option. This created a colour block with the value red, which I plugged into the socket to complete the action, as you can see in the lower part of the same image.

AIA Web application viewer area

I noted that the App Designer viewer/design canvas area is quite a bit ‘smaller’ than the actual screen area on my FreeRunner, which means that if I try to increase the height or width settings of components to take up more screen space on my phone, I run out of design canvas area in the App Designer. I expect this can be worked around by setting one or more dimensions to that of the screen, or a container, but I haven’t yet tried this out. It’s how GTK and other tool-kits work, though, so there should be a solution to this issue.

Name blocks versus value blocks

While setting up the canvas (in the PaintPot tutorial) to draw a circle on the Touched event, I first encountered event parameters. The blue blocks (*name x*, *name y*, *name touchedSprite*) dock in nicely with the event-handler block (green, *when DrawingCanvas.Touched*) as shown in Figure 10.

You might expect to be able to click-drag or somehow copy those name blocks and use them as parameters inside the event-handler block—but like me, you’d be wrong. In this case, after I added the *call DrawingCanvas.DrawCircle* block as the action to perform, I tried to drag the *name* blocks over and use them. You can’t. You have to pop open the *My Definitions* drawer under the *My Blocks* tab, and drag the *value x* and *value y* blocks down—and plug them into the *x* and *y* parameters for the *DrawCircle* method. As you can see in Figure 10, the parameter name (in bold) on the name block is the same as the parameter name (in bold) on the value block—that’s how you know they’re the same parameter.

You can rename parameters: click on the parameter name, which is in bold (*x* for instance), and type a new name (say, *‘touched_x’*) and hit *Enter*. If you already have a *value* block corresponding to that parameter on your canvas, that *value* block’s name also changes to reflect the new parameter name.

My Definitions is global?

At present, all variable declarations are global. However, the App Inventor team has accepted a feature request for local variables, which should probably be implemented soon. What is mildly confusing, though, is that parameter block definitions in *My Definitions* are apparently also global. While defining the *when DrawingCanvas.Dragged* block (an event handler to draw lines as the user drags a finger or a stylus across the canvas), the now-familiar name blocks turned up for the parameters to the event-handler block, as shown in Figure 11.

By now we know that the value blocks for these parameters should be in *My Definitions*—and yes, they are. However, while you’re working on this block, the value